

Trabajo Final Integrador

Food Store - Gestion de Datos

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a
Distancia

Materia: Base de Datos I

Estudiante: Valentina Lucia Caro

Legajo: 18314

Fecha de entrega: Junio de 2026

Enlace al video explicativo: [Base de Datos I Trabajo Práctico Integrador | UTN TUPAD](#)

Índice

Introducción.....	3
Resumen ejecutivo.....	3
Reglas de negocio.....	3
Desarrollo del trabajo.....	4
Diagrama Entidad-Relación (DER).....	4
Modelo Relacional (MR).....	5
Decisiones de Diseño.....	6
Evidencias.....	9
Consultas y resultados.....	9
Verificaciones de consistencia.....	10
Comparación con y sin índices.....	11
Principio de privilegios mínimos.....	13
Concurrencia.....	14
ANEXO.....	17
ETAPA 1: Modelado y definición de Constraints.....	17
ETAPA 2: Generación y carga de datos masivos con SQL puro.....	18
ETAPA 3: Consultas Avanzadas y Reportes.....	20
ETAPA 4: Seguridad e Integridad.....	22
ETAPA 5: Concurrencia y Transacciones.....	23

Introducción

Resumen ejecutivo

Este Trabajo Integrador de Bases de Datos refleja el diseño, implementación y optimización de una base de datos relacional a partir de un modelo de dominio. Se desarrollaron tareas relacionadas con el modelado físico, aplicación de restricciones de integridad y poblamiento de datos para realizar pruebas de rendimiento. Además, se trabajaron consultas avanzadas, uso de índices, políticas de seguridad y gestión de transacciones y concurrencia. El objetivo principal es aplicar los conocimientos de la materia en un contexto similar a una situación real, usando herramientas de inteligencia artificial como apoyo para la resolución del mismo.

Reglas de negocio

Para asegurar la integridad, consistencia y seguridad de la base de datos, se definieron las siguientes reglas de negocio:

1. Estructura común de auditoría: Las tablas principales del sistema (Usuario, Pedido, DetallePedido, Producto y Categoría) deben incluir los atributos heredados de la entidad Base: un identificador único (id), una fecha de creación automática (createdAt) y un indicador de eliminación lógica (eliminado).
2. Restricciones de unicidad: No se permite la duplicación de datos en campos críticos. Para ello se aplican restricciones UNIQUE sobre el atributo mail en la tabla usuario y nombre en la tabla categoría.
3. Restricciones de dominio: Se implementan restricciones CHECK para evitar datos inconsistentes. Los atributos precio y stock de producto deben ser mayores o iguales a cero, mientras que cantidad en DetallePedido debe ser mayor a cero.
4. Integridad referencial: Todas las relaciones entre tablas deben respetarse mediante claves foráneas. Cada pedido debe estar asociado a un usuario existente, cada producto a una categoría válida y cada detalle de pedido a un pedido y producto registrados.
5. Borrado lógico: Para conservar el historial de información, los registros no se eliminarán físicamente de la base de datos. En su lugar, se actualizará el atributo eliminado cambiando su valor a verdadero.
6. Consistencia de totales: El valor total de un pedido debe corresponder a la suma de los subtotales registrados en sus respectivos detalles.
7. Seguridad y control de acceso: El acceso a las tablas estará limitado mediante permisos específicos y el uso de vistas que permitan ocultar información sensible.
8. Prevención de inyecciones SQL: Las consultas dinámicas deberán utilizar consultas parametrizadas para reducir riesgos de seguridad.
9. Transacciones y concurrencia: Las operaciones que involucren múltiples registros, como la creación de un pedido con sus detalles, deberán ejecutarse dentro de transacciones para garantizar la atomicidad y mantener la consistencia de los datos.

Desarrollo del trabajo

Diagrama Entidad-Relación (DER)

En la Figura 1 se puede observar el Diagrama de Entidad-Relación conceptual realizado mediante el modelo de Chen, en el cual cada elemento se representa utilizando distintas formas geométricas de la siguiente manera:

- Entidades (Usuario, Pedido, Producto, DetallePedido y Categoria): rectángulos.
- Atributos (se seleccionaron únicamente los principales de cada entidad para mejorar la legibilidad del diagrama): óvalos, con las claves primarias (PK) subrayadas.
- Relaciones (1:N): rombos.

Por otro lado, en la Figura 2 se puede observar un Diagrama de Entidad-Relación físico más completo y detallado, realizado con la herramienta MySQL Workbench, utilizando el método Reverse Engineering para crear el diagrama automáticamente utilizando el *script del Modelo Relacional. En este caso se muestran las entidades con todos sus atributos y sus respectivos tipos de datos, además de las claves primarias (Primary Keys), claves foráneas (Foreign Keys) y las relaciones entre entidades, representadas mediante flechas punteadas (non-identifying relationship).

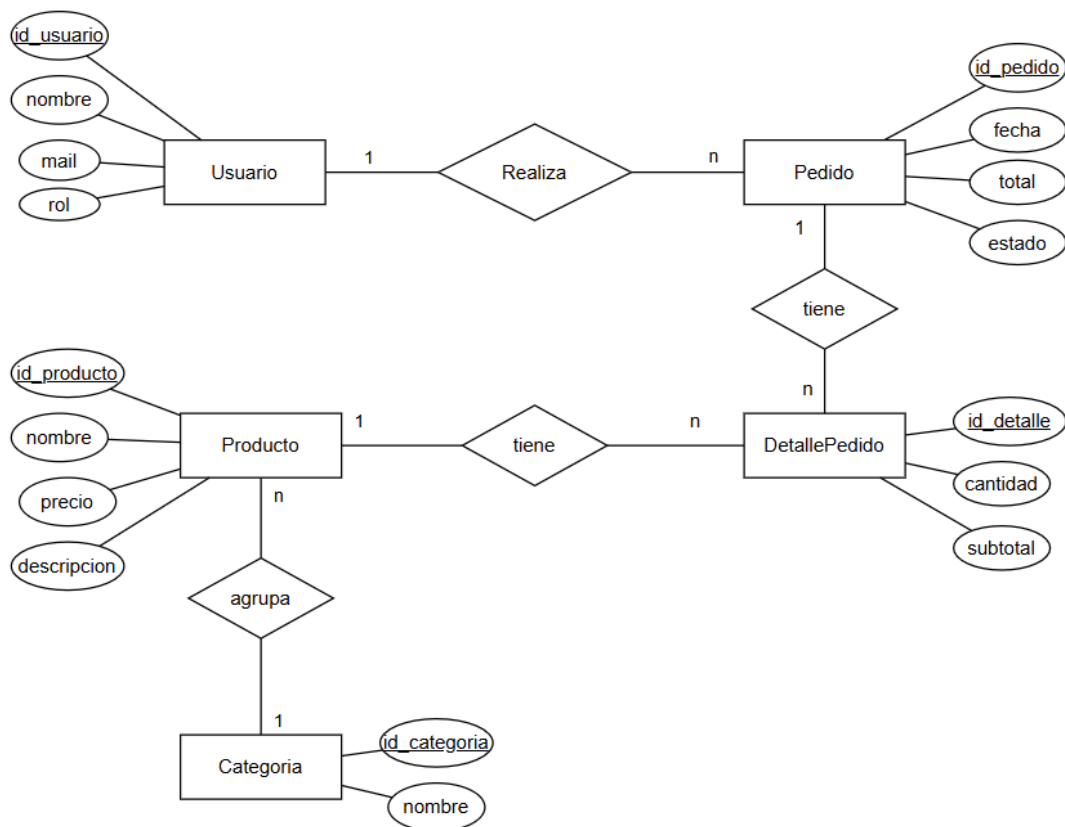


Figura 1: Diagrama Entidad-Relación (DER) con modelo de Chen, realizado con la herramienta draw.io

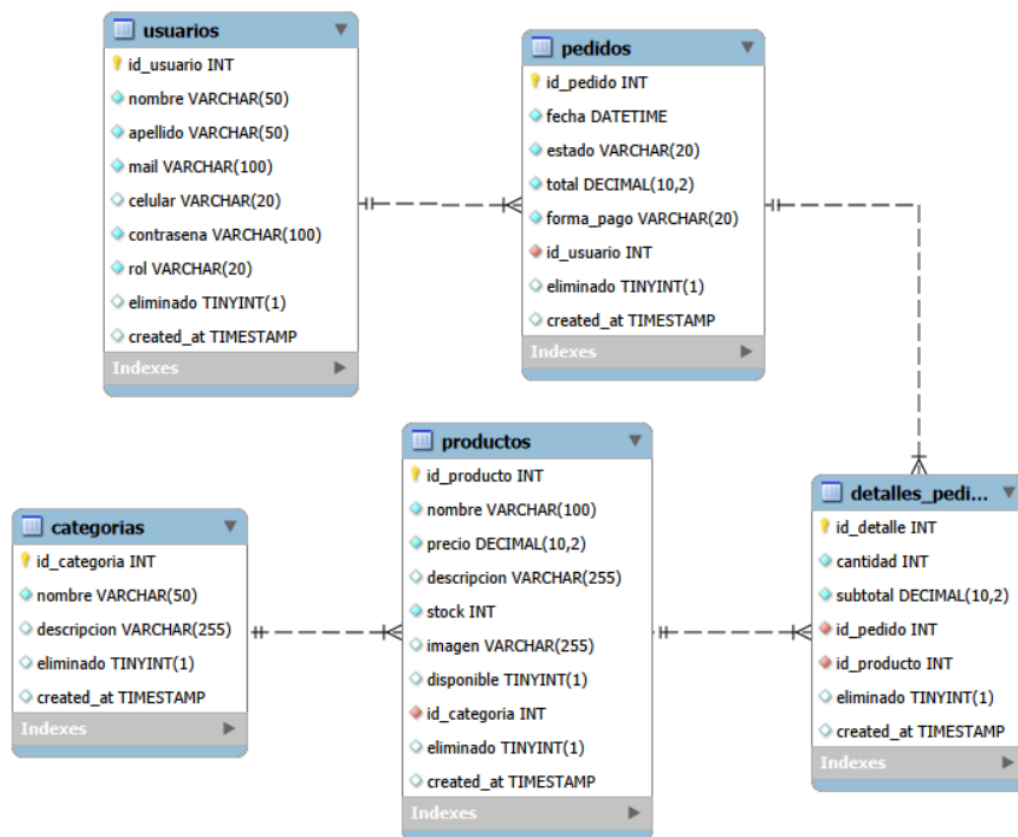


Figura 2: Diagrama Entidad-Relación (DER) físico realizado en MySQL Workbench

*Ver script completo utilizado en 01_esquema.sql

Modelo Relacional (MR)

-USUARIOS (id_usuario, nombre, apellido, mail, celular, contrasena, rol, eliminado, created_at)

-CATEGORIAS (id_categoria, nombre, descripcion, eliminado, created_at)

-PRODUCTOS (id_producto, nombre, precio, descripcion, stock, imagen, disponible, id_categoria, eliminado, created_at)

- Donde PRODUCTOS.id_categoria es FK que hace referencia a CATEGORIAS(id_categoria).

-PEDIDOS (id_pedido, fecha, estado, total, forma_pago, id_usuario, eliminado, created_at)

- Donde PEDIDOS.id_usuario es FK que hace referencia a USUARIOS(id_usuario).

-DETALLES_PEDIDO (id_detalle, cantidad, subtotal, id_pedido, id_producto, eliminado, created_at)

- Donde DETALLES_PEDIDO.id_pedido es FK que hace referencia a PEDIDOS(id_pedido).
- Donde DETALLES_PEDIDO.id_producto es FK que hace referencia a PRODUCTOS(id_producto).

Ver script 01_esquema.sql para la ejecución física de las tablas y constraints.

Decisiones de Diseño

Etapa 1:

El diseño del modelo relacional para el sistema Food Store se realizó a partir del diagrama de clases UML proporcionado por la cátedra, adaptando la lógica de Programación Orientada a Objetos a un esquema físico en SQL. Los atributos compartidos de la clase Base (id, eliminado y created_at) se incorporaron en las tablas principales para mantener una estructura uniforme.

Además, se utilizaron claves primarias numéricas con AUTO_INCREMENT para identificar cada registro y mantener la estabilidad de las relaciones entre tablas. También se implementaron restricciones como PRIMARY KEY, FOREIGN KEY, UNIQUE y CHECK con el objetivo de garantizar la integridad de los datos, evitar duplicados y validar rangos permitidos. Finalmente, se incorporó el mecanismo de borrado lógico (Soft Delete) mediante el atributo eliminado, permitiendo conservar el historial de información sin eliminar registros físicamente.

Ver 01_esquema.sql con la creación de las tablas. Pruebas de inserción en pruebas.sql.

Etapa 2:

Para cumplir con los objetivos planteados en la materia y poder realizar pruebas más cercanas a una situación real, fue necesario trabajar con un volumen considerable de información dentro de la base de datos. El poblado masivo se planificó con el objetivo de generar aproximadamente 190.000 registros distribuidos entre las distintas entidades del sistema Food Store. Contar con una cantidad importante de datos resulta fundamental para etapas posteriores del trabajo, especialmente aquellas relacionadas con la optimización y el análisis del rendimiento, ya que permite observar de manera más realista el comportamiento del motor de base de datos frente a consultas complejas e índices.

Para realizar esta tarea se decidió utilizar Expresiones de Tabla Comunes Recursivas (CTE Recursivas) mediante la instrucción WITH RECURSIVE, aprovechando herramientas proporcionadas directamente por MySQL. Se eligió esta alternativa porque permite generar grandes cantidades de registros desde el propio motor de la base de datos, evitando depender de programas externos o de estructuras repetitivas implementadas en otros lenguajes.

El funcionamiento de una CTE recursiva se basa en dos partes principales. La primera es el caso base o miembro ancla, que establece el punto inicial de la secuencia; por ejemplo, comenzando desde un valor inicial como SELECT 1 AS n. La segunda corresponde al miembro recursivo, encargado de generar nuevos valores tomando como referencia el resultado anterior, por ejemplo mediante incrementos sucesivos ($n + 1$). El proceso continúa automáticamente hasta alcanzar una condición de corte definida previamente.

En este caso, se utilizaron límites específicos según la cantidad de registros necesarios para cada entidad. Por ejemplo, para la generación de pedidos se trabajó con 40.000 registros y para la tabla DetallesPedido se generaron alrededor de 150.000 registros. De esta manera fue posible crear un conjunto de datos suficientemente amplio para realizar pruebas posteriores.

Durante la implementación surgió una limitación propia de MySQL relacionada con la profundidad máxima permitida para las CTE recursivas. De manera predeterminada, el

sistema establece un límite de 1000 iteraciones como medida de seguridad para evitar bucles infinitos y un uso excesivo de recursos. Sin embargo, debido a que el volumen requerido superaba ampliamente ese valor, fue necesario modificar temporalmente el parámetro `cte_max_recursion_depth`, incrementándolo a 250000 para permitir que el proceso pudiera completarse correctamente.

Otro aspecto importante durante la generación de datos fue mantener la integridad referencial entre las tablas. Para ello se utilizó el operador MOD, que permitió distribuir automáticamente las claves foráneas dentro de rangos válidos.

La fórmula utilizada fue:

$$FK = (\text{MOD}(n, M)) + 1$$

Donde n representa el valor generado por la secuencia y M corresponde a la cantidad total de registros existentes en la tabla principal. Por ejemplo, si la tabla Usuarios contiene 1000 registros, la operación genera valores comprendidos entre 1 y 1000, garantizando que cada pedido se asocie a un usuario existente y evitando errores por claves foráneas inválidas.

También fue necesario resolver un problema relacionado con la tabla DetallesPedido. Debido a que existe una restricción que impide registrar un mismo producto repetido dentro de un pedido, una asignación simple utilizando únicamente MOD podía provocar registros duplicados. Para evitar este inconveniente se incorporó una combinación de división entera mediante FLOOR y desplazamientos en la secuencia generada. Esto permitió distribuir los productos de manera más uniforme y mantener variedad en los pedidos creados.

Como resultado final, el sistema logró generar datos consistentes y sin conflictos de integridad, distribuyendo aproximadamente 150.000 registros en los detalles de pedido y alcanzando un promedio cercano a 3,7 productos por compra. Además de cumplir con los requisitos del trabajo, esta estrategia permitió construir una base de datos más realista para realizar pruebas de rendimiento y optimización en las etapas siguientes.

Ver `02_catalogo.sql` con la generación automática de usuarios y productos, y `03_carga_masiva.sql` con la generación automática de los registros. Control de volumen y auditorías en `prueba.sql`.

Etapas 3:

Para mejorar la organización y el mantenimiento del proyecto, se trabajó con distintos archivos SQL separados según su función, como índices, consultas, vistas y pruebas. Esta estructura permitió mantener una mejor organización y facilitar futuras modificaciones sin afectar otros componentes del sistema.

Las consultas avanzadas se diseñaron utilizando operaciones como JOIN, GROUP BY, HAVING y subconsultas, con el objetivo de obtener información útil a partir de los datos almacenados. Además, se implementaron índices para optimizar el rendimiento y reducir los tiempos de respuesta de las consultas más utilizadas. Finalmente, se crearon vistas para simplificar el acceso a la información y aplicar reglas de negocio, como el filtrado de registros eliminados lógicamente.

ver `04_indices.sql`, `05_consultas.sql`, `05_explain.sql` y `06_vistas.sql` con cada uno de estos scripts. Pruebas y comparaciones en `pruebas.sql`

Etapa 4

En esta etapa se implementaron diferentes medidas para reforzar la seguridad y el control de acceso dentro del sistema Food Store. Para ello, se aplicó el principio de mínimos privilegios mediante la creación de un usuario específico con permisos limitados, evitando el uso de cuentas administrativas y restringiendo el acceso directo a las tablas principales.

Además, se utilizaron vistas para simplificar el acceso a la información y proteger datos sensibles, como las contraseñas de los usuarios o registros que fueron eliminados lógicamente. También se incorporó un procedimiento almacenado para centralizar las consultas y mejorar la seguridad frente a posibles intentos de inyección SQL. Finalmente, se mantuvo la política de borrado lógico (Soft Delete), permitiendo conservar el historial de datos y mantener la integridad de las relaciones entre tablas.

Prueba de inyección SQL

Sentencia utilizada:

```
CALL sp_buscar_usuario_por_email('operador@foodstore.com' OR '1'='1');
```

Resultado obtenido:

Empty set (0.00 sec) → No se devolvieron registros.

Explicación:

La consulta maliciosa no produjo resultados porque el valor ingresado fue tratado como un dato y no como parte de la estructura de la consulta SQL. Al utilizar parámetros en el procedimiento almacenado y evitar la construcción dinámica de consultas mediante concatenación, el motor interpretó toda la cadena ingresada como un correo electrónico literal. Como no existe ningún registro con ese valor exacto, la consulta devuelve cero resultados, evitando así una posible inyección SQL y protegiendo la información almacenada.

ver 07_seguridad.sql con las restricciones y privilegios y prueba.sql con violaciones de integridad.

Etapa 5

En esta etapa se trabajó con el manejo de transacciones y la concurrencia dentro del sistema para asegurar la consistencia de los datos. Se decidió implementar el control transaccional mediante procedimientos almacenados en la base de datos, permitiendo centralizar la lógica y garantizar un manejo más seguro de las operaciones.

Además, se incorporó un mecanismo de control de errores y reintentos para gestionar posibles bloqueos entre transacciones (deadlocks), evitando fallos durante la ejecución simultánea de procesos. También se analizaron distintos niveles de aislamiento, como READ COMMITTED y REPEATABLE READ, seleccionando este último para las operaciones principales debido a que ofrece una mayor estabilidad y evita inconsistencias en los datos durante accesos concurrentes.

ver 09_concurrencia_guiada

Evidencias

Consultas y resultados

```

39 • SELECT c.nombre AS categoria, COUNT(dp.id_detalle) AS total_items_vendidos, SUM(dp.subtotal) AS recaudacion_total
40 FROM Categorias c
41 INNER JOIN Productos p ON p.id_categoria = c.id_categoria
42 INNER JOIN Detalles_Pedido dp ON dp.id_producto = p.id_producto
43 INNER JOIN Pedidos ped ON dp.id_pedido = ped.id_pedido
44 WHERE ped.estado = 'TERMINADO' AND ped.eliminado = FALSE
45 GROUP BY c.id_categoria, c.nombre
46 HAVING SUM(dp.subtotal) > 50000.00;
47

```

categoria	total_items_vendidos	recaudacion_total
Almacenamiento	12400	3567687500.00
Componentes de PC	12400	3343050000.00
Periféricos	12700	3456800000.00

Figura 3: Agregación de Ventas (GROUP BY + HAVING)
Ver 05_consultas.sql

```

52 • SELECT id_producto, nombre, precio, stock
53 FROM Productos
54 WHERE precio > (SELECT AVG(precio) FROM Productos WHERE eliminado = FALSE)
55 AND eliminado = FALSE;
56

```

id_producto	nombre	precio	stock
1	Monitor Gamer 24 FHD v1	225000.00	11
2	SSD NVMe 1TB Kingston v2	187500.00	12
3	Procesador AMD Ryzen v3	150000.00	13
7	Procesador AMD Ryzen v7	237500.00	17
8	Teclado Mecánico LOQ RGB v8	200000.00	18
9	Monitor Gamer 24 FHD v9	162500.00	19
13	Monitor Gamer 24 FHD v13	250000.00	23
14	SSD NVMe 1TB Kingston v14	212500.00	24
15	Procesador AMD Ryzen v15	175000.00	25
20	Teclado Mecánico LOQ RGB v20	225000.00	30
21	Monitor Gamer 24 FHD v21	187500.00	31
22	SSD NVMe 1TB Kingston v22	150000.00	32
26	SSD NVMe 1TB Kingston v26	237500.00	36
27	Procesador AMD Ryzen v27	200000.00	37
28	Teclado Mecánico LOQ RGB v28	162500.00	38
32	Teclado Mecánico LOQ RGB v32	250000.00	42
33	Monitor Gamer 24 FHD v33	212500.00	43
34	SSD NVMe 1TB Kingston v34	175000.00	44
39	Procesador AMD Ryzen v39	225000.00	49
40	Teclado Mecánico LOQ RGB v40	187500.00	50
41	Monitor Gamer 24 FHD v41	150000.00	51
45	Monitor Gamer 24 FHD v45	237500.00	55
46	SSD NVMe 1TB Kingston v46	200000.00	56
47	Procesador AMD Ryzen v47	162500.00	57
51	Procesador AMD Ryzen v51	250000.00	61
52	Teclado Mecánico LOQ RGB v52	212500.00	62
53	Monitor Gamer 24 FHD v53	175000.00	63
58	SSD NVMe 1TB Kingston v58	225000.00	68
59	Procesador AMD Ryzen v59	187500.00	69
60	Teclado Mecánico LOQ RGB v60	150000.00	70
64	Teclado Mecánico LOQ RGB v64	237500.00	74
65	Monitor Gamer 24 FHD v65	200000.00	75
66	SSD NVMe 1TB Kingston v66	162500.00	76
70	SSD NVMe 1TB Kingston v70	250000.00	80
71	Procesador AMD Ryzen v71	212500.00	81
72	Teclado Mecánico LOQ RGB v72	175000.00	82
77	Monitor Gamer 24 FHD v77	225000.00	87
78	SSD NVMe 1TB Kingston v78	187500.00	88
79	Procesador AMD Ryzen v79	150000.00	89
83	Procesador AMD Ryzen v83	237500.00	93
84	Teclado Mecánico LOQ RGB v84	200000.00	94
85	Monitor Gamer 24 FHD v85	162500.00	95
89	Monitor Gamer 24 FHD v89	250000.00	99
90	SSD NVMe 1TB Kingston v90	212500.00	100
91	Procesador AMD Ryzen v91	175000.00	10
96	Teclado Mecánico LOQ RGB v96	225000.00	15
97	Monitor Gamer 24 FHD v97	187500.00	16
98	SSD NVMe 1TB Kingston v98	150000.00	17
NULL	NULL	NULL	NULL

Figura 4: Análisis de precios (Subconsulta)
Ver 05_consultas.sql

```

1 • EXPLAIN ANALYZE
2 SELECT ped.id_pedido, u.nombre, prod.nombre
3 FROM Pedidos ped
4 INNER JOIN Usuarios u ON ped.id_usuario = u.id_usuario
5 INNER JOIN Detalles_Pedido dp ON dp.id_pedido = ped.id_pedido
6 INNER JOIN Productos prod ON dp.id_producto = prod.id_producto
7 WHERE ped.fecha BETWEEN '2026-01-01' AND '2026-06-30';

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

EXPLAIN

-> Nested loop inner join (cost=84.6 rows=11.1) (actual time=0.113..806 rows=127023 loops=1)...

Figura 5: Explain Analyze
Ver 05_explain.sql

Verificaciones de consistencia

```

1 -- control de volumen total
2 • SELECT 'Usuarios' AS Tabla, COUNT(*) AS Total_Registros FROM Usuarios
3 UNION ALL
4 SELECT 'Categorías', COUNT(*) FROM Categorías
5 UNION ALL
6 SELECT 'Productos', COUNT(*) FROM Productos
7 UNION ALL
8 SELECT 'Pedidos', COUNT(*) FROM Pedidos
9 UNION ALL
10 SELECT 'Detalles de Pedido', COUNT(*) FROM Detalles Pedido;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Tabla	Total_Registros
Usuarios	1002
Categorías	3
Productos	100
Pedidos	40000
Detalles de Pedido	150000

Figura 6: Registros de todas las tablas.
Ver pruebas.sql, etapa 2 (control de volumen total)

```

3 • SELECT COUNT(*) AS Productos_Huerfanos
4 FROM Productos
5 WHERE id_categoria NOT IN (SELECT id_categoria FROM Categorias);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:
Productos_Huerfanos
0

8 • SELECT COUNT(*) AS Pedidos_Huerfanos
9 FROM Pedidos
10 WHERE id_usuario NOT IN (SELECT id_usuario FROM Usuarios);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:
Pedidos_Huerfanos
0

13 • SELECT COUNT(*) AS Detalles_Huerfanos
14 FROM Detalles_Pedido
15 WHERE id_pedido NOT IN (SELECT id_pedido FROM Pedidos)
16 OR id_producto NOT IN (SELECT id_producto FROM Productos);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:
Detalles_Huerfanos
0

```

Figura 7: Verificación de elementos huérfanos (0 = no hay elementos huérfanos)
Ver pruebas.sql, etapa 2 (auditoría de integridad referencial)

Comparación con y sin índices

Rendimiento Rango Con índice	Rendimiento Rango Sin índice
<pre> 1 -- medicion con indices 2 -- rendimiento rango 3 SELECT ped.id_pedido, u.nombre, prod.nombre 4 FROM Pedidos ped 5 INNER JOIN Usuarios u ON ped.id_usuario = u.id_usuario 6 INNER JOIN Detalles_Pedido dp ON dp.id_pedido = ped.id_pedido 7 INNER JOIN Productos prod ON dp.id_producto = prod.id_producto 8 WHERE ped.fecha BETWEEN '2026-01-01 00:00:00' AND '2026-01-03 23:59:59' 9 AND ped.eliminado = FALSE; </pre> Query Statistics Timing (as measured at client side): Execution time: 0:00:0.09300000 Timing (as measured by the server): Execution time: 0:00:0.25767180 Table lock wait time: 0:00:0.00000400	<pre> 1 -- medicion sin indices 2 -- rendimiento rango 3 SELECT ped.id_pedido, u.nombre, prod.nombre 4 FROM Pedidos ped IGNORE INDEX (idx_pedido_fecha_total) 5 INNER JOIN Usuarios u ON ped.id_usuario = u.id_usuario 6 INNER JOIN Detalles_Pedido dp ON dp.id_pedido = ped.id_pedido 7 INNER JOIN Productos prod ON dp.id_producto = prod.id_producto 8 WHERE ped.fecha BETWEEN '2026-01-01 00:00:00' AND '2026-01-03 23:59:59' 9 AND ped.eliminado = FALSE; </pre> Query Statistics Timing (as measured at client side): Execution time: 0:00:0.14000000 Timing (as measured by the server): Execution time: 0:00:0.33205650 Table lock wait time: 0:00:0.00000400
Query Statistics Timing (as measured at client side): Execution time: 0:00:0.10900000 Timing (as measured by the server): Execution time: 0:00:0.28720080 Table lock wait time: 0:00:0.00000300	Query Statistics Timing (as measured at client side): Execution time: 0:00:0.11000000 Timing (as measured by the server): Execution time: 0:00:0.29225520 Table lock wait time: 0:00:0.00000400

Query Statistics Timing (as measured at client side): Execution time: 0:00:0.12500000 Timing (as measured by the server): Execution time: 0:00:0.30609490 Table lock wait time: 0:00:0.00000400	Query Statistics Timing (as measured at client side): Execution time: 0:00:0.15600000 Timing (as measured by the server): Execution time: 0:00:0.38390930 Table lock wait time: 0:00:0.00000500
Media measured by the server: 0.2836	Media measured by the server: 0.3360

Rendimiento JOIN Con índice	Rendimiento JOIN Sin índice
<pre> 1 -- medicion con indices 2 -- rendimiento JOIN 3 • SELECT p.id_producto, p.nombre, c.nombre 4 FROM Productos p 5 INNER JOIN Categorías c ON p.id_categoria = c.id_categoria 6 WHERE p.eliminado = FALSE; </pre> Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00077730 Table lock wait time: 0:00:0.00000400	<pre> 1 -- medicion sin indices 2 -- rendimiento JOIN 3 • SELECT p.id_producto, p.nombre, c.nombre 4 FROM Productos p IGNORE INDEX (idx_producto_categoria) 5 INNER JOIN Categorías c ON p.id_categoria = c.id_categoria 6 WHERE p.eliminado = FALSE; </pre> Query Statistics Timing (as measured at client side): Execution time: 0:00:0.01600000 Timing (as measured by the server): Execution time: 0:00:0.00063500 Table lock wait time: 0:00:0.00000400
Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00048700 Table lock wait time: 0:00:0.00000300	Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00055630 Table lock wait time: 0:00:0.00000400
Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00057830 Table lock wait time: 0:00:0.00000400	Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00068880 Table lock wait time: 0:00:0.00000400
Media measured by the server: 0.00061	Media measured by the server: 0.00062

Rendimiento Igualdad Con índice	Rendimiento Igualdad Sin índice
---------------------------------	---------------------------------

<pre> 1 -- medicion con indices 2 -- rendimiento igualdad 3 • SELECT id_producto, nombre, precio, stock 4 FROM Productos 5 WHERE id_categoria = 2 AND eliminado = FALSE; </pre> Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00037680 Table lock wait time: 0:00:0.00000200	<pre> 1 -- medicion sin indices 2 -- rendimiento igualdad 3 • SELECT id_producto, nombre, precio, stock 4 FROM Productos IGNORE INDEX (idx_producto_categoria) 5 WHERE id_categoria = 2 AND eliminado = FALSE; </pre> Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00052010 Table lock wait time: 0:00:0.00000400
Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00056400 Table lock wait time: 0:00:0.00000300	Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00055900 Table lock wait time: 0:00:0.00000300
Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00058230 Table lock wait time: 0:00:0.00000300	Query Statistics Timing (as measured at client side): Execution time: 0:00:0.00000000 Timing (as measured by the server): Execution time: 0:00:0.00051140 Table lock wait time: 0:00:0.00000300
Media measured by the server: 0.00050	Media measured by the server: 0.00053

Principio de privilegios mínimos

```

1  -- acceso permitido
2  -- el operador tiene permisos SELECT sobre las vistas seguras
3  •  SELECT * FROM tfi_base_datos.v_usuarios_seguros LIMIT 5;
4
5

```

	id_usuario	nombre	apellido	mail	celular	rol	created_at
▶	1	Valentina	Caro	valentina@mail.com	3364123456	ADMIN	2026-06-19 15:55:25
	2	Agustin	Farris	agustin@mail.com	3364654321	USUARIO	2026-06-19 15:55:25
	3	Agustin	Farris	usuario_1@mail.com	3364000001	ADMIN	2026-06-19 15:55:25
	4	Mariana	Gomez	usuario_2@mail.com	3364000002	ADMIN	2026-06-19 15:55:25
	5	Carlos	Lopez	usuario_3@mail.com	3364000003	ADMIN	2026-06-19 15:55:25

```

6  -- el operador tiene permisos INSERT/UPDATE sobre Categorías
7  • INSERT INTO tfi_base_datos.Categorías (nombre, descripción)
8  VALUES ('Audio Profesional', 'Sistemas de sonido de alta fidelidad');
9

```

#	Time	Action	Message
1	16:55:43	SELECT * FROM tfi_base_datos.v_usuarios_seguros LIMIT 5	5 row(s) returned
2	16:57:19	INSERT INTO tfi_base_datos.Categorías (nombre, descripción) VALUES ('Audio Profesional', 'Sistemas de sonido de alta fidelidad');	1 row(s) affected

Figura 8: accesos permitidos para el usuario operador_foodstore ver 07_seguridad.sql (usuario con permisos acotados) y practica.sql, etapa 4 (pruebas con el usuario)

```

10 -- acceso restringido
11 -- intento malicioso de leer las contraseñas directamente de la tabla base 'Usuarios'
12 • SELECT id_usuario, mail, contrasena FROM tfi_base_datos.Usuarios;
13
14 -- intento de borrar físicamente un producto
15 • DELETE FROM tfi_base_datos.Productos WHERE id_producto = 1;

```

#	Time	Action	Message
1	16:55:43	SELECT * FROM tfi_base_datos.v_usuarios_seguros LIMIT 5	5 row(s) returned
2	16:57:19	INSERT INTO tfi_base_datos.Categorías (nombre, descripción) VALUES ('Audio Profesional', 'Sistemas de sonido de alta fidelidad');	1 row(s) affected
3	17:01:25	SELECT id_usuario, mail, contrasena FROM tfi_base_datos.Usuarios LIMIT 0, 1000	Error Code: 1142. SELECT command denied to user 'operador_foodstore'@'localhost' for table 'usuarios'
4	17:01:37	DELETE FROM tfi_base_datos.Productos WHERE id_producto = 1	Error Code: 1142. DELETE command denied to user 'operador_foodstore'@'localhost' for table 'productos'

Figura 9: accesos denegados para el usuario operador_foodstore ver 07_seguridad.sql (usuario con permisos acotados) y practica.sql, etapa 4 (pruebas con el usuario)

Concurrencia

Se realizó la siguiente prueba en dos consolas diferentes para intentar realizar cambios en el mismo elemento al mismo tiempo (ver pruebas.sql (prueba deadlock))

En la Figura 9 se puede ver el resultado obtenido en ambas consolas; al intentar hacer el último UPDATE, la segunda consola lanza la excepción de Deadlock found.

✓	159	13:55:17	START TRANSACTION	0 row(s) affected
✓	160	13:55:26	UPDATE Productos SET stock = 10 WHERE id_producto = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	161	13:56:06	UPDATE Productos SET stock = 15 WHERE id_producto = 2	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
✓	5	13:56:00	START TRANSACTION	0 row(s) affected
✓	6	13:56:02	UPDATE Productos SET stock = 20 WHERE id_producto = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✗	7	13:56:13	UPDATE Productos SET stock = 25 WHERE id_producto = 1	Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction

Figura 9: Error code: Deadlock found

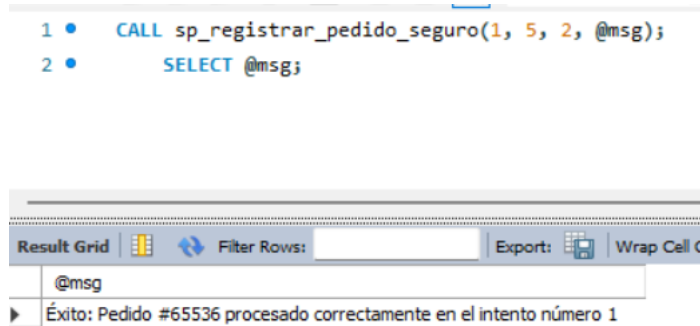
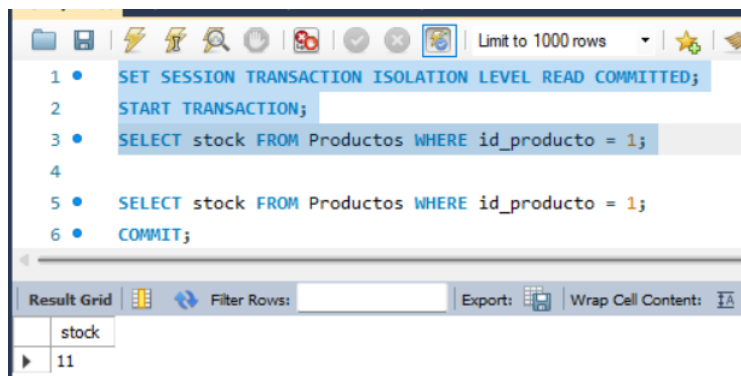


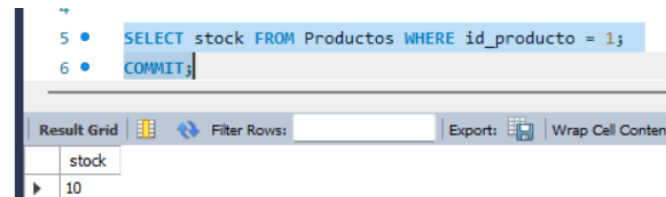
Figura 10: Manejo de deadlocks
ver 09_concurrencia_guiada.sql



terminal A

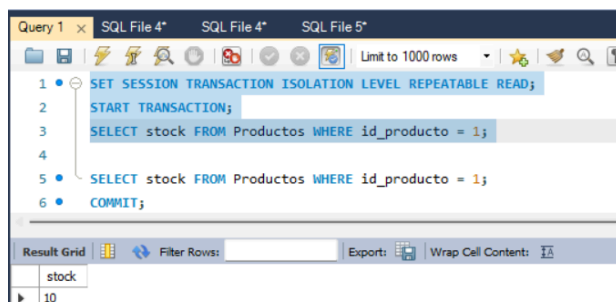
#	Time	Action	Message
27	15:55:54	START TRANSACTION	0 row(s) affected
28	15:55:54	UPDATE Productos SET stock = 10 WHERE id_producto = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
29	15:55:54	COMMIT	0 row(s) affected

terminal B



terminal A

Figura 11: read committed



terminal A

#	Time	Action	Message
✓ 30	16:02:00	START TRANSACTION	0 row(s) affected
✓ 31	16:02:00	UPDATE Productos SET stock = 99 WHERE id_producto = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 32	16:02:00	COMMIT	0 row(s) affected

terminal B

5	•	SELECT stock FROM Productos WHERE id_producto = 1;
6	•	COMMIT;

stock
99

terminal A

Figura 12: repeatable read

ANEXO

Para la realización de este trabajo se utilizó como herramienta de apoyo la Inteligencia Artificial Gemini (Google DeepMind), empleando el prompt sugerido en el anexo de las consignas. En dicho prompt se le solicita a la IA actuar como un tutor, evitando proporcionar respuestas completas y directas, y funcionando más bien como una herramienta de guía y aprendizaje. De esta manera, se busca que el estudiante pueda razonar, analizar y corregir su propio trabajo a partir de la orientación recibida.

ETAPA 1: Modelado y definición de Constraints

Prompt:

Hola. Estoy diseñando el modelo relacional para mi TFI de Bases de Datos I. Para las entidades Usuario, Categoría, Producto, Pedido y DetallePedido, definí como Clave Primaria (PK) el atributo id. Antes de pasarlo a SQL, quiero validar con vos si esta elección de PK es robusta o si ves algún riesgo de redundancia o inconsistencia a futuro.

Respuesta generada:

¡Qué buen avance! Elegir un identificador subrogado (como un **id** numérico autoincremental) como Clave Primaria para tus tablas principales (**Usuario**, **Categoría**, **Producto** y **Pedido**) es una decisión excelente y muy alineada con las mejores prácticas de la industria.

Hacer esto te da una gran robustez porque independiza la identidad de la fila de los datos de negocio. Por ejemplo, si un usuario cambia su correo electrónico o una categoría cambia de nombre, las relaciones (claves foráneas) no se rompen porque apuntan a un **id** inmutable. Además, para los **JOIN** que vas a tener que hacer más adelante en la Etapa 3, comparar valores numéricos es muchísimo más eficiente para el motor de la base de datos. Sin embargo, para asegurarnos de que tu diseño sea completamente blindado contra la redundancia e inconsistencia antes de escribir el SQL, te propongo que analicemos juntos un punto clave:

El caso de **DetallePedido** (¿Clave subrogada o compuesta?)

En tu diseño de objetos, **DetallePedido** hereda el **id** de la clase **Base**. Pero en el modelo relacional, vale la pena detenerse a pensar: **DetallePedido** es una parte de una composición (depende enteramente de un **Pedido**).

Si usás un **id** propio como PK, la base de datos te va a permitir, por ejemplo, registrar estas dos filas:

- Fila 1: **id: 101**, **pedido_id: 5**, **producto_id: 20**, **cantidad: 2**
- Fila 2: **id: 102**, **pedido_id: 5**, **producto_id: 20**, **cantidad: 1**

El motor de la base de datos no va a chistar porque los PKs (**101** y **102**) son distintos. Pero a nivel de negocio, ¿tiene sentido que el mismo producto aparezca dos veces como líneas separadas en el mismo pedido? Si querés evitar esta inconsistencia, tenés dos caminos:

1. Mantener el **id** único como PK y añadir una restricción **UNIQUE(pedido_id, producto_id)**.
2. Eliminar el **id** independiente y transformar la PK de **DetallePedido** en una **clave compuesta** por **(pedido_id, producto_id)**.

ETAPA 2: Generación y carga de datos masivos con SQL puro

Prompt:

Tengo que poblar mi base de datos con un volumen masivo de registros ficticios (aproximadamente entre 10.000 y 200.000 filas) usando solo SQL puro. Quiero asegurar que las Claves Foráneas (FK) siempre apunten a valores válidos y respeten las cardinalidades del negocio. Estaba leyendo sobre usar tablas semilla con producto cartesiano o secuencias con WITH RECURSIVE. ¿Podrías explicarme conceptualmente los pros y contras de estas estrategias para mi caso, y darme un ejemplo mínimo y genérico (sin resolverme mi script) de cómo se estructuraría la lógica?

Respuesta generada:

¡Excelente que ya estés encarando la **Etapa 2**! Vamos a analizar las dos estrategias que mencionás para que elijas la que mejor se adapte a tu diseño.



Estrategia 1: Tablas Semilla + Producto Cartesiano

(CROSS JOIN)

Esta técnica consiste en crear tablas temporales o variables pequeñas con datos reales (por ejemplo, una lista de 50 nombres y otra de 50 apellidos) y multiplicarlas recursivamente mediante un **CROSS JOIN**.

Pros:

- **Realismo en los datos:** Combinar 100 nombres con 100 apellidos te da inmediatamente 10.000 combinaciones de personas con un aspecto muy natural, evitando el clásico y aburrido "Usuario 1", "Usuario 2".
- **Rendimiento óptimo:** Los motores de bases de datos son extremadamente rápidos realizando productos cartesianos en memoria.

Contras:

- **Crecimiento exponencial explosivo:** Si te descuidás con el tamaño de las semillas, podés pasar de 10.000 a 10.000.000 de registros en un segundo, por lo que exige controlar el flujo con cláusulas como **LIMIT**.
- **Dificultad con patrones secuenciales:** No es la mejor herramienta si necesitás que un campo incremente de uno en uno de forma estrictamente ordenada o para simular fechas consecutivas.



Estrategia 2: Secuencias con WITH RECURSIVE (CTEs)

Un *Common Table Expression* (CTE) recursivo funciona conceptualmente como un bucle **while** o **for** tradicional, pero ejecutado directamente dentro del motor de la base de datos.

Pros:

- **Control absoluto del volumen:** Podés indicarle al motor exactamente de qué número a qué número querés que itere (por ejemplo, de 1 a 50.000), facilitando alcanzar el volumen planificado.
- **Ideal para datos técnicos y fechas:** Es la estrategia perfecta para construir tablas de tiempo o calendarios (sumando días consecutivamente a una fecha inicial).

Contras:

- **Monotonía visual:** Generar texto a partir de un número secuencial resulta en datos artificiales (como "correo" + id + "@mail.com").

- **Límites de recursividad:** Dependiendo del motor (como MySQL), existe un límite físico de recursión por defecto (generalmente 1.000 iteraciones) que requiere modificar variables de sesión (`max_sp_recursion_depth` o `cte_max_recursion_depth`) para poder alcanzar las 100.000 filas.

El Secreto de las FK: El Operador Matemático

Modulo (MOD)

Para cumplir con la condición de que **todas las FK apunten a PKs válidas** sin tener que escribir IDs a mano, el mejor truco matemático en SQL es combinar tu secuencia numérica con el operador **Módulo (% o MOD)**.

Si tenés exactamente **10 registros** en una tabla padre y necesitás asociarle **1.000 registros** en una tabla hija, la fórmula matemática determinista para la FK es:

$id_padre = (id_hijo \text{ (mod } 10)) + 1$

Esto garantiza que, sin importar qué tan grande sea el ID del hijo, el resultado de la FK siempre oscilará estrictamente entre 1 y 10, eliminando el riesgo de claves huérfanas.

 Ejemplo Mínimo y Genérico

Para ver cómo se estructura esta lógica de manera limpia y modular sin resolver tu caso particular, imaginemos un modelo genérico de dos tablas: **Autores** (Padre) y **Libros** (Hijo). Primero, asumamos que la tabla **Autores** ya fue poblada y tiene IDs autoincrementales del 1 al 50. El siguiente script utiliza un bloque recursivo para fabricar miles de libros y asociarlos de forma segura a esos autores:

SQL

```
-- 1. Definimos la secuencia lógica en memoria
WITH RECURSIVE SecuenciaLibros AS (
    -- Caso base: Empezamos en el registro número 1
    SELECT 1 AS id_hijo

    UNION ALL

    -- Caso recursivo: Incrementamos de a uno hasta el límite deseado
    SELECT id_hijo + 1
    FROM SecuenciaLibros
    WHERE id_hijo < 25000 -- Cambiando este límite controlás el volumen masivo
)

-- 2. Insertamos el resultado de la secuencia en la tabla definitiva
INSERT INTO Libros (titulo, sinopsis, autor_id, precio)
SELECT
    CONCAT('Libro Fantástico Volumen ', id_hijo),
```

```

    CONCAT('Esta es la sinopsis detallada del volumen de prueba
número ', id_hijo),

    -- TRUCO DE INTEGRIDAD: Supongamos que tenemos exactamente 50
    autores cargados.
    -- El módulo de cualquier número respecto a 50 da entre 0 y
    49. Sumando 1, garantizamos
    -- que autor_id siempre sea un entero válido entre 1 y 50.
    (id_hijo % 50) + 1,

    -- Introducimos pseudoaleatoriedad para que los precios
    varíen con sentido de negocio
    ROUND(1000 + (id_hijo * 7.5 % 5000), 2)
FROM SecuenciaLibros;

```

Con este enfoque mixto, la secuencia maneja la escala del volumen , mientras que la aritmética del módulo blindo por completo la integridad referencial de tus relaciones, evitando que salten errores de FK inválidas durante la carga masiva.

ETAPA 3: Consultas Avanzadas y Reportes

Prompt:

Estoy diseñando una consulta avanzada para un reporte de ventas mensuales por categoría. Mi idea inicial es usar un JOIN entre las tablas A y B, y agrupar con GROUP BY y un filtro HAVING. Como voy a manejar un volumen masivo de datos, quiero aprender a analizar su rendimiento. ¿Qué herramientas conceptuales (como EXPLAIN) o qué índices me sugerís evaluar para revisar si la consulta es eficiente?

Respuesta generada:

Diseñar reportes avanzados que procesen volúmenes masivos de datos es el escenario perfecto para entender cómo sufre —o disfruta— el motor de una base de datos. Cuando combinás JOIN, GROUP BY y HAVING, el orden en que el motor ejecuta las operaciones y la existencia de caminos pavimentados (índices) lo cambian todo.

A continuación, te presento las herramientas conceptuales y las estrategias de indexación clave para auditar y pulverizar los tiempos de ejecución de tu consulta.

1. Herramientas de Diagnóstico Técnico

EXPLAIN y EXPLAIN ANALYZE

Antes de medir tiempos, tenés que mirar el "mapa de ruta" que el optimizador de la base de datos dibuja para tu consulta. EXPLAIN te muestra la estrategia estimada antes de ejecutarla, mientras que EXPLAIN ANALYZE (disponible en motores modernos como MySQL 8.0+) ejecuta la consulta de verdad y te devuelve el costo y el tiempo real en milisegundos de cada paso.

Al revisar el árbol de ejecución, prestá atención a estos indicadores críticos:

- **Tipo de acceso (Access Type):** Si ves la palabra **ALL**, significa que el motor está haciendo un *Full Table Scan* (leyendo toda la tabla de arriba a abajo en el disco). Buscá que mude a **ref**, **eq_ref** o **range**.
- **Filas examinadas (Rows):** Compará la cantidad de filas que el motor *estima* inspeccionar contra las que realmente devuelve. Si para traerte 12 filas de un reporte mensual tiene que leer 200.000, hay una ineficiencia enorme.
- **Operaciones pesadas:** Alerta máxima si encontrás carteles como **Using temporary** (el motor crea una tabla fantasma en memoria/disco para procesar tus datos) o **Using filesort** (el motor tiene que reordenar los registros en caliente porque no pudo aprovechar un índice para el **GROUP BY**).

2. Estrategias de Indexación Eficientes

Para un reporte que une tablas, filtra temporalmente y agrupa por categorías, los índices simples a veces se quedan cortos. Evaluá las siguientes variantes:

Índices en Claves Foráneas (FK)

El primer cuello de botella ocurre en el **JOIN**. Si estás uniendo la Tabla A con la Tabla B a través de una clave foránea (por ejemplo, **producto_id** o **categoria_id**), **esa columna FK debe tener un índice**. Sin él, por cada fila de la Tabla A, el motor tendrá que escanear la Tabla B completa para encontrar la coincidencia.

Índices Compuestos (Multi-columna)

Para reportes con agregaciones, el orden de las columnas dentro del índice altera drásticamente el resultado. El motor procesa las consultas en un orden lógico estricto: primero filtra (**WHERE**), luego junta (**JOIN**) y después agrupa (**GROUP BY**).

Si creás un **índice compuesto** que siga exactamente ese orden anatómico, el motor puede hacer lo que se conoce como un *Index Scan* ordenado, matando el **Using filesort** de raíz.

Ejemplo de diseño: Si filtrás por rango de fechas y agrupás por categoría, un índice estructurado como (**fecha**, **categoria_id**) le permite al motor recortar el universo de datos por el mes específico y, de forma inmediata, encontrar los datos ya ordenados por categoría en la estructura interna de la base de datos.

El rol del **HAVING**

Recordá que el filtro **WHERE** actúa *antes* de agrupar y el **HAVING** actúa *después* (sobre los resultados agregados, como un **SUM** o un **COUNT**). Los índices **no ayudan directamente al HAVING** porque este opera sobre datos calculados al vuelo. Si podés mudar alguna condición del **HAVING** al **WHERE** (por ejemplo, filtrar un ID de categoría en lugar de hacerlo después de agrupar), hacelo; aliviará el volumen de datos que entra al proceso de agrupación.

3. Metodología de Medición en Entornos Reales

Para que tus conclusiones tengan valor analítico y científico al armar tu informe técnico, aplicá estas pautas metodológicas:

- **Evitá el sesgo de la caché:** La primera vez que ejecutás una consulta pesada puede tardar 500ms porque va al disco, pero la segunda puede tardar 5ms porque quedó guardada en la memoria caché del motor. Para medir el rendimiento real, ejecutá la consulta al menos 3 veces y tomá como métrica estable **la mediana de los tiempos**.

- **Prueba de contraste (A/B Testing):** Para demostrar el impacto real de tu optimización, medí el tiempo de la consulta obligando al motor a ignorar el índice (usando cláusulas como **IGNORE INDEX** o directamente eliminándolo temporalmente con **DROP INDEX**). Luego, volví a medir forzando el camino optimizado si es necesario con **FORCE INDEX**. La diferencia entre ambas medianas será tu mejor evidencia de eficiencia.

ETAPA 4: Seguridad e Integridad

Prompt:

Para la etapa de seguridad, creé un usuario con privilegios mínimos y diseñé dos vistas para ocultar información sensible en las tablas. En la capa de aplicación (Java con JDBC), quiero asegurarme de implementar consultas parametrizadas seguras para evitar SQL Injection. ¿Qué tipo de pruebas de inserción inválida o entradas maliciosas me recomendás diseñar para verificar físicamente que mis constraints y mi código están bloqueando los ataques?

Respuesta generada:

¡Excelente! Necesitás diseñar y documentar dos tipos de pruebas físicas: las de **integridad de datos** a nivel de motor de base de datos y las de **anti-inyección** en tu capa de persistencia en Java con JDBC.

A continuación, te presento el diseño estratégico de los escenarios de prueba que podés ejecutar:



Part I: Pruebas de Integridad Física (Constraints de Base de Datos)

Estas pruebas demuestran que tu diseño relacional aborta transacciones inválidas de forma nativa si alguien intenta saltarse la lógica de la aplicación.

Prueba 1: Violación de Unicidad (**UNIQUE**)

- **Objetivo:** Comprobar que el motor impide la duplicación de datos de identidad cruciales, como el correo del usuario.
- **Entrada Inválida (SQL directo):** Intentar un **INSERT** en la tabla de usuarios utilizando un correo electrónico idéntico a uno que cargaste previamente en la carga masiva.
- **Resultado Esperado:** El motor debe rechazar inmediatamente la sentencia y arrojar un código de error específico de clave duplicada (por ejemplo, el código **1062** en MySQL para *Duplicate entry*).

Prueba 2: Violación de Restricciones de Rango (**CHECK**)

- **Objetivo:** Garantizar que los estados financieros o de inventario mantengan coherencia de negocio (valores no negativos).
- **Entrada Inválida (SQL directo):** Ejecutar un **INSERT** o un **UPDATE** en la tabla de productos asignando un **precio = -150.00** o un **stock = -5**.
- **Resultado Esperado:** Al contar con una restricción **CHECK (precio >= 0 AND stock >= 0)**, el motor interrumpirá la ejecución devolviendo un error de violación de restricción check (código **3819** en MySQL).

Part II: Pruebas de Entrada Maliciosa (SQL Injection en Java)

Cuando usás consultas parametrizadas, el controlador JDBC envía la estructura de la consulta al motor para que se compile **antes** de fusionarla con los datos ingresados por el operador. Al ingresar el payload, el motor no lo interpreta como comandos lógicos, sino como una cadena de texto literal (un simple string inofensivo).

Podés diseñar estas dos entradas simuladas desde tu menú de consola para capturar el comportamiento seguro del sistema:

Ataque 1: Evasión por Tautología (Bypass de Filtros)

Imaginá un submenú de consulta donde el operador ingresa un correo para buscar un cliente específico en la base de datos.

- **Entrada Maliciosa:** En la consola, cuando el programa solicite el correo, ingresá exactamente:
- `operador@foodstore.com' OR '1'='1`
- **Lo que intentaría un SQL dinámico concatenado:** Modificar la estructura lógica para transformar el filtro en una condición siempre verdadera (**true**), exponiendo o listando todos los usuarios de la base de datos sin restricciones.
- **Comportamiento Seguro con JDBC:** Al usar parámetros posicionales (**?**), el motor buscará textualmente un usuario cuyo correo sea literalmente el string `"operador@foodstore.com' OR '1'='1"`. Como ningún usuario tiene ese correo real, el sistema simplemente retornará un mensaje controlado indicando que la entidad no fue encontrada, demostrando que la inyección fue completamente neutralizada.

Ataque 2: Inyección de Consultas Apiladas (Stacked Queries Destructivas)

Imaginá que estás editando el nombre de una categoría mediante el ID de la misma.

- **Entrada Maliciosa:** Cuando el menú te pida ingresar el nuevo nombre, introducí un payload destructivo:
- `Pizzas'; DROP TABLE Pedido; --`
- **Lo que intentaría un SQL dinámico concatenado:** Cerrar la comilla de la instrucción original, finalizar la sentencia con un punto y coma (;) y concatenar un comando destructivo para borrar la tabla de pedidos históricos.
- **Comportamiento Seguro con JDBC:** El **PreparedStatement** escapará automáticamente los caracteres especiales. La base de datos modificará el nombre de la categoría a la cadena de texto literal `"Pizzas'; DROP TABLE Pedido; --"`. Tu tabla de pedidos permanecerá intacta y a salvo, y podrás comprobar la seguridad listando nuevamente las categorías para ver el curioso nombre de texto que se guardó.

ETAPA 5: Concurrencia y Transacciones

Prompt:

Logré simular un deadlock ejecutando de forma simultánea dos sesiones en mi SGBD que intentaban actualizar los mismos registros en orden cruzado. Necesito implementar un mecanismo de reintento (retry con backoff). ¿Podrías guiarme conceptualmente en los

pasos lógicos que debería seguir mi código para capturar este error específico y reintentar la transacción de forma segura?

Respuesta generada:

¡Felicitaciones por lograr simular el *deadlock*! Tu objetivo ahora es lograr que la aplicación reaccione con elegancia, aplicando **hasta 2 reintentos con un backoff breve**.

Para diseñar este mecanismo de forma robusta, te presento la secuencia de pasos lógicos y la estructura arquitectónica que debería seguir tu código.



Flujo Lógico del Mecanismo de Reintento

El secreto de este patrón es tratar a toda la transacción como un bloque de ejecución atómico y repetible dentro de un bucle controlado.

1. Inicialización del Bucle de Intentos

Antes de tocar la base de datos, debés definir las variables de control del bucle:

- **Límite de intentos:** En tu caso, un máximo de 3 ejecuciones totales (el intento original + los 2 reintentos solicitados por la cátedra).
- **Contador actual:** Empezando en 1.
- **Variable de éxito:** Un indicador booleano inicializado en `false`.

2. El Bloque Seguro (Try-Catch)

Dentro de cada iteración del bucle, debés estructurar el manejo de la transacción de la siguiente manera:

- **Paso A: Apertura Limpia de la Transacción** Iniciá la transacción explícitamente (`START TRANSACTION` o configurando el `autoCommit` en falso si lo hacés desde código de aplicación).
- **Paso B: Ejecución de la Lógica de Negocio** Corré tus sentencias SQL secuencialmente (las lecturas, las actualizaciones de los registros, etc.).
- **Paso C: Confirmación (COMMIT)** Si todas las líneas se ejecutaron sin problemas, llamá al `COMMIT` para consolidar los datos, marcá tu variable de éxito como `true` y rompé el bucle (`break`). ¡Flujo feliz completado!

3. Captura y Filtrado del Error (El Bloque Catch)

Si algo falla durante el Paso B, el flujo saltará inmediatamente al bloque de excepciones.

Aquí es donde ocurre la magia de la seguridad:

- **Paso D: El Rollback Inmediato (Obligatorio)** Lo primero que debe hacer tu código al capturar *cualquier* error es ejecutar un `ROLLBACK`. Esto es vital porque, si fuiste la víctima del *deadlock*, necesitás liberar inmediatamente los pocos candados (locks) que tu sesión haya alcanzado a retener, permitiendo que la otra sesión ganadora termine su trabajo y libere el resto de los recursos.
- **Paso E: Identificación del Error Técnico** Inspeccioná el error capturado. Debés validar mediante condicionales si el código de error numérico devuelto por el SGBD es exactamente `1213` o si el string de `SQLSTATE` es `40001`.
 - Si *NO* es un *deadlock*: Registrá el error en tu log y cortá el bucle lanzando la excepción hacia arriba. No tiene sentido reintentar un error de sintaxis o una violación de `FOREIGN KEY`.
 - Si *SÍ* es un *deadlock*: Procedé al cálculo del reintento.

4. Evaluación de Intentos y Aplicación del Backoff

- **Paso F: Verificar el Límite** Si tu contador actual ya llegó al máximo de intentos permitidos, significa que el sistema está bajo una contención extrema. Registrá el fallo definitivo en el log, lanzá la excepción y notificá al usuario.
- **Paso G: Pausa Estratégica (Backoff)** Si todavía te quedan reintentos disponibles, **no vuelvas a ejecutar la transacción de inmediato**. Si lo hacés, lo más probable es que choques contra el mismo hilo competidor que todavía está procesando sus datos.
- Poné a "dormir" el hilo de ejecución actual durante unos milisegundos (por ejemplo, entre 100ms y 300ms). Para hacerlo aún más profesional, podés agregarle una pizca de aleatoriedad (*jitter*), asegurando que si dos hilos sufrieron el deadlock al mismo tiempo, no se vuelvan a despertar exactamente en el mismo instante.
- **Paso H: Siguiendo Iteración** Incrementá tu contador de intentos en 1 y dejá que el bucle vuelva a comenzar desde el **Paso A**.